

پروژه نظریه اطلاع و کد گذاری

استاد رضا سبحانی
سیده زهرا طباطبایی
شماره دانشجویی: ۴۰۲۴۰۱۳۰۶۱

فهرست مطالب

۲	۱ مقدمه و اهداف پروژه
۲	۲ راهنمای گام به گام ورودی ها و خروجی های انتظار رفته
۴	۳ سورس کد کامل پروژه (Python)

۱ مقدمه و اهداف پروژه

هدف از این پروژه، پیاده‌سازی و شبیه‌سازی گام‌به‌گام یک زنجیره مخابراتی کامل دیجیتال شامل بخش‌های زیر است:

- کدگذاری منبع (Source): (Coding) فشرده‌سازی داده‌ها با استفاده از الگوریتم هافن پویا بر روی منبع توسعه‌یافته S^L .
- کدگذاری کانال (Channel): (Coding) حفاظت از بیت‌ها در برابر نویز با استفاده از کدهای ساختاریافته و پویای Hamming (n, k) سیستماتیک.
- شبیه‌سازی کانال: اعمال نویز با استفاده از مدل کانال باینری متقارن (BSC).
- ارزیابی عملکرد: رسم نمودار نرخ خطای بیت (P_e) بر حسب نسبت سیگنال به نویز (SNR) و مقایسه آن با مرز تئوری حد شانون.

۲ راهنمای گام‌به‌گام ورودی‌ها و خروجی‌های انتظار رفته

گام اول: (تولید منبع، معتبرسازی و کدگذاری هافن پویا) - گزینه ۱

در این گام، مشخصات آماری منبع پیام دریافت شده، صحت ریاضی احتمالات بررسی می‌شود و سپس کدهای بهینه هافن برای منبع توسعه‌یافته تولید می‌گردند.
ورودی‌های انتظار رفته: (Inputs)

- symbols source of Number (q): تعداد نمادهای الفبای منبع (مثلاً ۳ یا ۴).
- Probabilities: احتمال وقوع هر نماد به صورت عدد اعشاری (مثلاً ۰.۴۰، ۰.۳۰، ۰.۲۰، ۰.۱۰).
- L order Extension: مرتبه توسعه منبع (عدد ۱ برای نمادهای تک‌تک یا ۲ برای بسته‌های دوتایی).

خروجی‌های انتظار رفته: (Outputs)

- در صورت بروز خطا: اگر مجموع احتمالات دقیقاً برابر ۱ نشود یا احتمالی خارج از بازه $[0, 1]$ باشد، پیام خطای Error Validation چاپ شده و برنامه فوراً بسته می‌شود.
- در صورت موفقیت: جدول نگاشت (Codebook) کدهای باینری اختصاص یافته به هر نماد یا ترکیب نمادها (بلاک‌ها) در ترمینال چاپ می‌شود. (مثلاً: Block, 'S1', 'S1' -> Binary Code: 00)

گام دوم: (تولید ماتریس‌های پویا و سیستماتیک همینگ) - گزینه ۲

این گام وظیفه ساخت ساختار ریاضی که کنترل خطای همینگ را بر عهده دارد، ویژگی مهم آن، پویایی (Dynamic) بودن بر اساس نیاز کاربر است.
ورودی‌های انتظار رفته: (Inputs)

• r bits parity of Number: تعداد بیت‌های توازن که همینگ (مثلاً عدد ۳ برای که همینگ استاندارد یا ۴ برای مرتبه بالاتر).

خروجی‌های انتظار رفته: (Outputs)

• در صورت بروز خطا: اگر کاربر عددی کمتر از ۲ وارد کند یا فرمت متنی بنویسد، خطای معتبرسازی صادر شده و برنامه قطع می‌شود.

• در صورت موفقیت: ابعاد دقیق که شامل طول کل بلاک (n) و تعداد بیت‌های داده (k) به همراه نرخ که ($Rate = k/n$) محاسبه شده و دو ماتریس مولد سیستماتیک G و ماتریس ارزیابی توازن سیستماتیک H به فرم استاندارد $[A|I_r]$ در ترمینال چاپ می‌شوند.

گام سوم: (تست مستقل کانال باینری متقارن - BSC) - گزینه ۳

رفتار نویز تصادفی کانال BSC را روی یک رشته باینری دلخواه شبیه‌سازی می‌کند.
ورودی‌های انتظار رفته: (Inputs)

• string Binary: یک رشته از صفر و یک‌ها به عنوان دیتای فرستاده شده (TX) (مثلاً ۱۰۱۱۰۰۱).

• P_{bar} probability Crossover: احتمال جابجایی یا همان احتمال خطای کانال که باید عددی بین ۰ تا ۵۰۰ باشد (مثلاً ۱۰۰).

خروجی‌های انتظار رفته: (Outputs)

• در صورت بروز خطا: اگر کاراکتری غیر از ۰ و ۱ وارد شود (مثلاً ۴۴۴) یا احتمال خطا در بازه مجاز نباشد، برنامه خطای سخت گیرانه داده و بسته می‌شود.

• در صورت موفقیت: رشته ارسالی (TX)، (Bits رشته دریافت شده‌ی نویزی (RX) (Bits و تعداد دقیق بیت‌هایی که به صورت کاملاً تصادفی بر اثر نویز معکوس شده‌اند (Injected Errors) نمایش داده می‌شود.

گام چهارم: (زنجیره کامل شبیه‌سازی و رسم عملکرد) - گزینه ۴

مهم‌ترین بخش پروژه که تمام ۴ فاز قبلی را به صورت یک زنجیره متوالی (End-to-End) به هم متصل کرده و عملکرد سیستم را ارزیابی می‌کند.
ورودی‌های انتظار رفته: (Inputs)

• دریافت همزمان: تعداد نمادها (q)، بردار احتمالات (P)، طول کل رشته (N)، مرتبه توسعه (L) و تعداد بیت‌های توازن (r).

• شرط ویژه معتبرسازی: علاوه بر شروط قبلی، حتماً باید طول رشته (N) بر مرتبه توسعه (L) بخش پذیر باشد (باقی مانده صفر شود)، در غیر این صورت برنامه اجازه شبیه سازی نمی دهد.

خروجی های انتظار رفته: (Outputs)

- سیستم پس از اجرای محاسبات، یک نمودار گرافیکی (Chart Performance) باز می کند.
- در این نمودار محور افقی نسبت سیگنال به نویز (SNR) و محور عمودی نرخ خطای بیت (BER) است.
- عملکرد سیستم بدون کدگذاری (خط نارنجی) در کنار سیستم مجهز به کد همینگ (خط آبی) رسم شده و خط عمودی قرمز رنگ مرز شانون که به صورت پویا بر اساس نرخ کد محاسباتی شما حل شده است، به عنوان حد نهایی تئوری در میانه ی صفحه خودنمایی می کند.

۳ سورس کد کامل پروژه (Python)

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 import itertools
4 import sys
5
6 def generate_dynamic_huffman(probs_dict):
7     nodes = [[prob, [sym]] for sym, prob in probs_dict.items()]
8     while len(nodes) > 1:
9         nodes = sorted(nodes, key=lambda x: x[0])
10        left = nodes.pop(0)
11        right = nodes.pop(0)
12        for sym in left[1]:
13            item_code[sym] = '0' + item_code.get(sym, '')
14        for sym in right[1]:
15            item_code[sym] = '1' + item_code.get(sym, '')
16        combined_symbols = left[1] + right[1]
17        nodes.append([left[0] + right[0], combined_symbols])
18    return item_code
19
20 item_code = {}
21
22 def get_dynamic_huffman_dict(symbols, probabilities, L):
23     ext_symbols = list(itertools.product(symbols, repeat=L))
24     ext_probs = [np.prod(p) for p in itertools.product(probabilities,
25                                                         repeat=L)]
26     probs_dict = dict(zip(ext_symbols, ext_probs))
27     global item_code
```

```

27     item_code = {sym: '' for sym in ext_symbols}
28     generate_dynamic_huffman(probs_dict)
29     return item_code
30
31 def generate_hamming_matrices(r):
32     n = 2**r - 1
33     k = n - r
34     H_base = np.zeros((r, n), dtype=int)
35     for i in range(1, n + 1):
36         H_base[:, i-1] = [int(x) for x in format(i, f'0{r}b')]
37     id_cols = []
38     for i in range(r):
39         vec = np.zeros(r, dtype=int)
40         vec[i] = 1
41         for j in range(n):
42             if np.array_equal(H_base[:, j], vec):
43                 id_cols.append(j)
44                 break
45     remaining_cols = [x for x in range(n) if x not in id_cols]
46     A = H_base[:, remaining_cols]
47     I_r = H_base[:, id_cols]
48     H = np.hstack((A, I_r))
49     G = np.hstack((np.eye(k, dtype=int), A.T))
50     return G, H, n, k
51
52 def simulate_bsc(bits, p_err):
53     noise = np.random.rand(len(bits)) < p_err
54     return (bits + noise) % 2
55
56 def decode_hamming(received, H, n, k):
57     corrected = []
58     for i in range(0, len(received), n):
59         block = received[i:i+n]
60         if len(block) < n: break
61         syndrome = np.dot(block, H.T) % 2
62         if np.any(syndrome):
63             for col in range(n):
64                 if np.array_equal(H[:, col], syndrome):
65                     block[col] = (block[col] + 1) % 2
66                     break
67         corrected.extend(block[:k])
68     return np.array(corrected)
69
70 def main():
71     print("\n=====")
72     print("      INFORMATION THEORY & CODING SIMULATION SYSTEM      ")
73     print("=====")
74     print("1. Standalone Test: Phase 2 (Dynamic Huffman Codes)")
75     print("2. Standalone Test: Phase 3 (Dynamic Hamming Matrices)")
76     print("3. Standalone Test: Phase 4 (BSC Channel Noise)")

```

```

77 print("4. Run Full Chain Simulation & Plot Performance Chart")
78 print("=====")
79
80 choice = input("Please select an option (1-4): ")
81
82 if choice == '1':
83     try:
84         q = int(input("Enter number of source symbols (q): "))
85         if q <= 1:
86             print(" Validation Error: q must be > 1!")
87             sys.exit()
88         symbols = [f"S{i+1}" for i in range(q)]
89         probabilities = []
90         for i in range(q):
91             p_val = float(input(f" Enter probability for {
symbols[i]}: "))
92             if p_val < 0 or p_val > 1: raise ValueError
93             probabilities.append(p_val)
94             if not np.isclose(sum(probabilities), 1.0, atol=1e-5):
95                 print(" Validation Error: Sum must equal 1.0!")
96                 sys.exit()
97             L = int(input("Enter extension order L (1 or 2): "))
98             codebook = get_dynamic_huffman_dict(symbols,
probabilities, L)
99             for k, v in list(codebook.items())[:8]:
100                 print(f" Block {k} -> Code: {v}")
101         except ValueError:
102             print(" Validation Error: Invalid numeric input format!")
103     )
104     sys.exit()
105
106 elif choice == '2':
107     try:
108         r = int(input("Enter number of parity bits r: "))
109         if r < 2: sys.exit()
110         G, H, n, k = generate_hamming_matrices(r)
111         print(f" Hamming({n}, {k})\nMatrix G:\n", G, "\nMatrix H
:\n", H)
112     except ValueError:
113         sys.exit()
114
115 elif choice == '3':
116     try:
117         bit_str = input("Enter binary string: ")
118         if not all(b in ['0', '1'] for b in bit_str):
119             print(" Validation Error: Must be ONLY '0' and '1'!")
120     )
121     sys.exit()
122     bits = np.array([int(b) for b in bit_str])
123     p = float(input("Enter p (0 to 0.5): "))
124     if p < 0 or p > 0.5: sys.exit()

```

```

123         rx = simulate_bsc(bits, p)
124         print("TX:", bits, " RX:", rx, " Errors:", np.sum(bits !=
rx))
125     except ValueError:
126         sys.exit()
127
128     elif choice == '4':
129         try:
130             q = int(input("Enter number of source symbols (q): "))
131             symbols = [f"S{i+1}" for i in range(q)]
132             probabilities = [float(input(f" Probability {symbols[i
]}}: ")) for i in range(q)]
133             if not np.isclose(sum(probabilities), 1.0, atol=1e-5):
sys.exit()
134
135             N = int(input("Enter length N: "))
136             L = int(input("Enter extension L: "))
137             if N % L != 0:
138                 print(f" Validation Error: N ({N}) must be divisible
by L ({L}).")
139                 sys.exit()
140             r = int(input("Enter parity bits r: "))
141
142             codebook = get_dynamic_huffman_dict(symbols,
probabilities, L)
143             source_seq = np.random.choice(symbols, size=N, p=
probabilities)
144             binary_stream = "".join([codebook[tuple(source_seq[i:i+L
])] for i in range(0, len(source_seq), L)])
145             tx_bits = np.array([int(b) for b in binary_stream])
146
147             G, H, n, k = generate_hamming_matrices(r)
148             coded_bits = []
149             for i in range(0, len(tx_bits), k):
150                 block = tx_bits[i:i+k]
151                 if len(block) < k: block = np.pad(block, (0, k - len(
block)), 'constant')
152                 coded_bits.extend(np.dot(block, G) % 2)
153             coded_bits = np.array(coded_bits)
154
155             p_errors = np.linspace(0.005, 0.18, 12)
156             snr_axis, ber_uncoded, ber_coded = [], [], []
157             for p_sim in p_errors:
158                 snr_axis.append((1 - p_sim) / p_sim)
159                 ber_uncoded.append(np.mean(tx_bits != simulate_bsc(
tx_bits, p_sim)))
160                 rx_coded = simulate_bsc(coded_bits, p_sim)
161                 decoded = decode_hamming(rx_coded, H, n, k)
162                 min_len = min(len(tx_bits), len(decoded))
163                 ber_coded.append(max(np.mean(tx_bits[:min_len] !=
decoded[:min_len]), 1e-4))

```

```

164         R = k / n
165         low, high = 1e-6, 0.5 - 1e-6
166         p_star = 0.1
167         for _ in range(50):
168             mid = (low + high) / 2
169             h_p = -mid * np.log2(mid) - (1 - mid) * np.log2(1 -
170 mid)
171             if np.isclose(1 - h_p, R, atol=1e-4): p_star = mid;
172 break
173             elif (1 - h_p) > R: low = mid
174             else: high = mid
175         shannon_limit_snr = (1 - p_star) / p_star
176
177         plt.figure(figsize=(7, 5))
178         plt.yscale('log'); plt.xscale('log')
179         plt.plot(snr_axis, ber_uncoded, 'x-', color='orange',
180 label='Uncoded System')
181         plt.plot(snr_axis, ber_coded, 'o-', color='blue', label=f
182 'Hamming({n},{k})')
183         plt.axvline(x=shannon_limit_snr, color='red', linestyle='
184 --', label=f'Shannon Limit ({shannon_limit_snr:.2f})')
185         plt.xlabel('SNR (Log Scale)'); plt.ylabel('BER (Log Scale
186 )'); plt.grid(True, which="both", ls="--")
187         plt.legend(); plt.show()
188     except ValueError:
189         sys.exit()
190
191 if __name__ == "__main__":
192     main()

```

Listing 1: